Practical 9: Time Intelligence and Advanced DAX for RetailPulse

Related topic: Advanced Data Modeling, Time Intelligence, Complex DAX Functions, Performance Optimization (Unit 4)

Name:		Roll No.:		Date:	

Note on figures and data: the tables and visuals in this report were generated from a sample RetailPulse dataset (150 sales orders, 8 products, 8 stores, 4 regions, Jan 2024 - Dec 2025) built to match the lab manual's schema. They reproduce what Power BI Desktop shows for the steps. If your workbook differs, your numbers will differ; paste your own Power BI screenshots into the marked boxes before submitting. The Performance Analyzer timings in this report are illustrative; record your own durations.

Objective

Create a date table, build time-intelligence and context-modifying DAX measures, and use Performance Analyzer to find and reduce slow report elements.

Materials Required

Power BI Desktop and RetailPulse_Analysis.pbix from Practical 8. Sales data covers two calendar years (2024-2025), so the year-on-year measures show values for 2025.

Procedure and Observations

Step 1 - Date table. Modeling > New table, then add columns:

```
Date = CALENDAR(MIN(Sales[Sales Date]), MAX(Sales[Sales Date]))
Year = YEAR('Date'[Date])
Month Number = MONTH('Date'[Date])
Month Name = FORMAT('Date'[Date], "MMM")
```

Select **Month Name** > Column tools > Sort by column > **Month Number**, so months sort Jan to Dec. Right-click the Date table > **Mark as date table** > choose Date[Date]. In Model view drag **Date[Date]** to **Sales[Sales Date]** (one-to-many, single direction).

Step 2 - Time-intelligence measures.

```
Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])
Sales LY  = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
```

Step 3 - YoY Growth % with variables. VAR stores a value once, so the formula is readable and Sales LY is not evaluated twice:

```
YoY Growth % =
VAR Curr = [Total Sales]
VAR Prev = [Sales LY]
RETURN
    DIVIDE(Curr - Prev, Prev)
```

Format as Percentage (Measure tools > Format).

Step 4 - Region Share %. ALL removes the Region filter from the denominator, so each region is divided by the grand total:

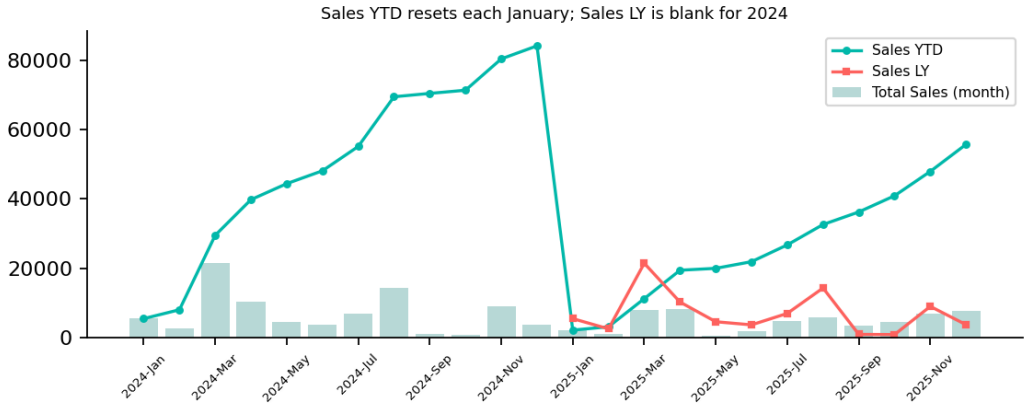
```
Region Share % = DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Store[Region])))
```

Step 5 - Matrix. Insert Matrix: Rows = Date[Year] > Date[Month Name]; Values = Total Sales, Sales YTD, Sales LY, YoY Growth %.

Year	Month	Total Sales	Sales YTD	Sales LY	YoY Growth %
2024	Jan	5,501	5,501		
2024	Feb	2,560	8,061		
2024	Mar	21,446	29,507		

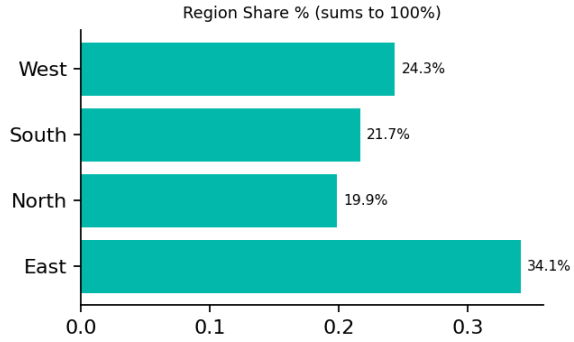
Year	Month	Total Sales	Sales YTD	Sales LY	YoY Growth %
2024	Apr	10,273	39,780		
2024	May	4,606	44,386		
2024	Jun	3,710	48,096		
2024	Jul	7,004	55,100		
2024	Aug	14,321	69,421		
2024	Sep	967	70,388		
2024	Oct	917	71,305		
2024	Nov	9,025	80,330		
2024	Dec	3,755	84,085		
2025	Jan	2,189	2,189	5,501	-60.2%
2025	Feb	970	3,159	2,560	-62.1%
2025	Mar	8,103	11,262	21,446	-62.2%
2025	Apr	8,185	19,447	10,273	-20.3%
2025	May	548	19,996	4,606	-88.1%
2025	Jun	1,914	21,910	3,710	-48.4%
2025	Jul	4,838	26,748	7,004	-30.9%
2025	Aug	5,866	32,614	14,321	-59.0%
2025	Sep	3,616	36,229	967	274.1%
2025	Oct	4,671	40,900	917	409.2%
2025	Nov	7,010	47,910	9,025	-22.3%
2025	Dec	7,799	55,710	3,755	107.7%

Observation: Sales YTD accumulates through each year and restarts in January 2025; 2024 rows have blank Sales LY and YoY Growth % because there is no earlier year; from January 2025 Sales LY equals the same month's Total Sales in 2024.

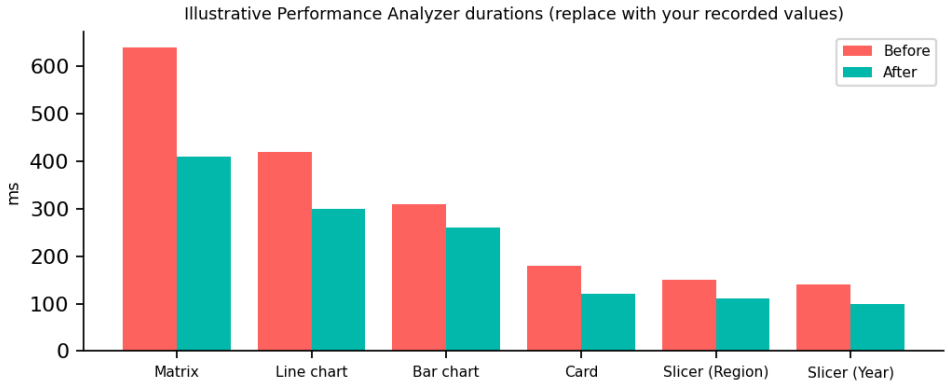


Region Share % (Store[Region] on rows of a table visual):

Region	Total Sales	Region Share %
East	47,708.85	34.1%
North	27,753.84	19.9%
South	30,294.89	21.7%
West	34,037.14	24.3%
Total	139,794.72	100.0%



Step 6 - Performance Analyzer. View > Performance analyzer > **Start recording** > **Refresh visuals**. Expand each visual to read DAX query, Visual display and Other times, and note the total duration. Optimisation done: removed unused columns from the model (Power Query), reduced the number of visuals on the page, replaced the high-cardinality Date field with Month Name in the matrix, and kept measures simple using variables. Record again and compare.



Visual	Before (ms)	After (ms)	Change made
Matrix	640	410	Fewer fields, variables in DAX
Line chart	420	300	Month instead of date
Bar chart	310	260	Removed unused columns
Card	180	120	-
Slicer (Region)	150	110	Single-select
Slicer (Year)	140	100	Single-select

Paste your Power BI screenshot here: Matrix with Sales YTD, Sales LY, YoY Growth %

Paste your Power BI screenshot here: Performance Analyzer before and after

Result / Expected Outcome Check

• Month rows reset the YTD value at the start of each year.

• First-year rows show blank Sales LY and YoY Growth %.

• Region Share % values add up to 100% (see table above).

• Performance Analyzer shows measurable timing for each visual and a clear record of what changed.

Viva-Voce Questions and Answers (related to this practical)

a) Why must a proper date table be marked as a date table?

Time-intelligence functions (TOTALYTD, SAMEPERIODLASTYEAR, DATEADD) need a continuous, gap-free date column with unique dates. Marking it tells Power BI which column to use and disables the hidden auto date/time tables, so results are correct and the model stays small.

b) What does CALCULATE do?

CALCULATE evaluates an expression after modifying the filter context, e.g. adding, replacing or removing filters. It is the core of context-modifying DAX; SAMEPERIODLASTYEAR and ALL are used as its filter arguments here.

c) Why use VAR and RETURN?

Variables are evaluated once and reused, which improves performance and readability and makes formulas easier to debug.

d) What is the role of ALL in Region Share %?

ALL removes the filters on Store[Region] from the denominator, so it always returns the grand total while the numerator remains filtered by the current region; the ratio is then the region's share.

e) What is Performance Analyzer used for?

It records how long each visual takes to refresh, split into DAX query, visual display and other time, so slow visuals or measures can be found and optimised.